LAB5 – DevOps

Equipe de Professores de TI

DevOps–LAB Mar/2026 – rev1

Apresentação

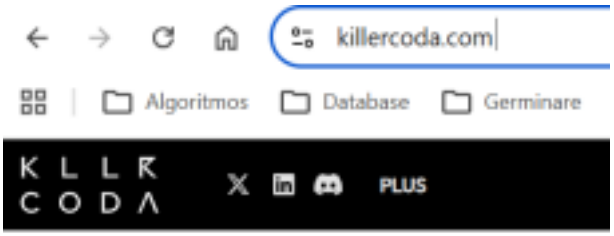
Esta lista de exercícios abrange os tópicos da Seção 3 do Guia de Referência Docker, com foco em:

- 3.1 — Criar e executar containers
- 3.2 — Controlar containers em execução
- 3.3 — Inspecionar e depurar containers

Os exercícios estão organizados em três níveis de dificuldade:

Nível	Descrição	Identificação
Básico	Comandos isolados e simples	Exercícios 1 a 4
Intermediário	Combinação de comandos	Exercícios 5 a 8
Avançado	Cenários integrados e análise	Exercícios 9

Observação: todos os exercícios podem ser realizados localmente com o Docker instalado ou em ambientes de laboratório online como o https://killercoda.com/



3.1 — Criar e Executar Containers

Nesta seção você praticará o uso do comando docker run e suas principais flags, aprendendo a iniciar containers de diferentes formas.

Exercício 1 — Meu Primeiro Container

Execute os comandos abaixo na ordem indicada e responda as perguntas.

a) Execute o comando abaixo:

```
docker run hello-world
```

b) Em seguida, execute:

```
docker run --rm ubuntu echo "Olá, Docker!"
```

Questões:

- O que aconteceu na primeira execução? O Docker precisou baixar alguma coisa? O docker tentou baixar a biblioteca Hello World, porém ele não conseguiu porque ela não existe

- O que significa a flag --rm no segundo comando?

Ela é responsável por excluir a container após executa-la

- Execute docker ps -a após os dois comandos. O container ubuntu ainda aparece? Por quê?

Não pois quando usamos o comando B, ele excluiu a nossa container

UNISENAI LAB5 – DevOps Equipe de Professores de TI
DevOps–LAB Mar/2026 – rev1

Exercício 2 — Modo Interativo vs. Modo Detached

Experimente os dois modos de execução abaixo:

Modo interativo:

```
docker run -it ubuntu bash
```

Dentro do container, execute os comandos: ls, pwd, hostname e depois exit.

Modo detached:

```
docker run -d --name meu-nginx nginx
```

Questões:

- Qual flag habilita o terminal interativo? O que cada letra significa?

O comando it, por ser responsável para enviar comandos para o container

- Qual a diferença entre executar em modo -it e em modo -d?

Quando se usa o comando -d ele roda em segundo plano, dessa forma não abrindo novos comandos no cmd.

- Como você verificaria que o container meu-nginx está realmente rodando?

Através do comando (docker ps -a)

Exercício 3 — Mapeamento de Portas (Desafio)

Execute um servidor nginx com a porta publicada no host:

```
docker run -d --name webserver -p 8080:80 nginx
```

Tente acessar http://localhost:8080 pelo navegador ou com o comando:

```
curl http://localhost:8080
```

Questões:

- O que significa o par 8080:80 no parâmetro -p?

É a atribuição do acesso 8080 do servidor na porta 80

- Se você usasse -p 9090:80, qual URL deveria acessar no navegador?

Teria que acessar (https://localhost:9090)

- É possível executar dois containers nginx mapeando a mesma porta do host (ex: duas vezes -p 8080:80)?

Não pois a porta do host só pode ser usado apenas uma vez

Exercício 4 — Variáveis de Ambiente

Suba um container MySQL passando variáveis de ambiente obrigatórias:

```
docker run -d --name meu-mysql \
 -e MYSQL_ROOT_PASSWORD=senhaSegura123 \
 -e MYSQL_DATABASE=escola \
 mysql:8.0
```

Questões:

• Para que serve a flag -e? Cite um exemplo de uso em uma aplicação real.

Ela é uma variavel do ambiente que permite passar as configurações para o container no momento que ele é utilizado.

EX:

docker run -d \

-e DB_HOST=mysql \

-e DB_USER=root \

-e DB_PASSWORD=senha123 \

minha-api

• Como você verificaria que as variáveis foram passadas corretamente ao container?

Através do comando (docker exec -it meu-mysql bash)

• Por que não é uma boa prática passar senhas diretamente via -e em produção?

Porque variaveis de ambiente podem ser facilmente expostas, quando se executa o comando para mostrar as variaveis.

3.2 — Controlar Containers em Execução

Nesta seção você aprenderá a gerenciar o ciclo de vida dos containers: iniciar, parar, pausar e remover.

Exercício 5 — Ciclo de Vida Completo

Execute a sequência de comandos abaixo e anote o resultado de cada etapa:

1. Criar e iniciar o container:
   `docker run -d --name ciclo-teste nginx`
2. Verificar o status:
   `docker ps`
3. Parar o container:
   `docker stop ciclo-teste`
4. Verificar novamente:
   `docker ps -a`
5. Iniciar novamente:
   `docker start ciclo-teste`
6. Remover o container (sem parar):
   `docker rm -f ciclo-teste`

Questões:

- Qual a diferença entre docker stop e docker kill?

O docker stop finaliza a sessão de forma segura, já o docker kill interrompe a sessão imediatamente

- Por que o container ainda aparece no docker ps -a após o docker stop?

Pois o stop não exclui o container, ele apenas encerra a execução

- O que acontece se você tentar docker rm em um container em execução sem a flag -f?

Ele irá dar erro de execução, onde ele não permite encerrar o container sem usar o comando stop

Exercício 6 — Pause e Unpause

Execute os comandos abaixo para observar o comportamento de pause/unpause:

```
docker run -d --name pausetest nginx
docker pause pausetest
docker ps
docker unpause pausetest
docker ps
```

Questões:

• O que acontece com os processos internos do container quando ele é pausado?

O docker irá suspender todos os processos dentro do container

• Como o status do container aparece no docker ps quando está pausado?

Aparecerá como UP(paused)

• Cite uma situação prática onde pausar um container seria útil.

-Liberar CPU temporariamente

- Utilizar antes de realizar um backup

Exercício 7 — Limpeza de Containers

Crie alguns containers para praticar a limpeza:

```
docker run --name ex7-a -d nginx
docker run --name ex7-b -d nginx
docker run --name ex7-c -d nginx
docker stop ex7-a ex7-b ex7-c
```

Agora experimente:

```
docker container prune
```

Questões:

- O que o comando docker container prune faz? Quais containers ele remove?

Ele remove todos os containers que estão parados. Desta forma removendo ex7-a, ex7-b, ex7-c

- Como você removeria de uma só vez todos os containers em execução? (Consulte o Cheat Sheet do guia)

Utilizando o comando docker rm -f $(docker ps -aq)

- Qual a diferença entre docker container prune e docker system prune?

O docker container prune remove apenas containers parados, enquanto o System pruine apaga instancias que estão paradas no sistema

Exercício 8 — Reinicialização e Restart Policy

Execute um container com política de reinicialização automática:

```
docker run -d --name auto-restart --restart=always nginx
```

Agora force a parada do processo interno:

```
docker exec auto-restart nginx -s stop
```

Aguarde alguns segundos e verifique:

```
docker ps
```

Questões:

• O que significa a política --restart=always?

Faz com que ele reinicie o container toda vez que ele parar

• O container reiniciou automaticamente após o nginx ser parado? O que isso indica?

Sim, indicando que a politica restart funciona corretamente

• Quais outras políticas de restart existem? Quando usaria --restart=on-failure?

--restart=no / --restart=always / --restart=unless-stopped.

Deve ser utilizada em aplicações que não podem falhar, desta forma sendo reiniciada toda vez que acabar caindo

3.3 — Inspecionar e Depurar Containers

Nesta seção você utilizará ferramentas de monitoramento e depuração para investigar o estado interno de containers.

Exercício 9 — Diagnóstico Completo de um Container

Inicie um container e realize uma investigação completa:

```
docker run -d --name diagnostico -p 8090:80 nginx
```

Execute cada comando abaixo e observe a saída:

1. Verificar os logs:
   `docker logs diagnostico`
2. Inspecionar detalhes (JSON):
   `docker inspect diagnostico`
3. Ver processos internos:
   `docker top diagnostico`
4. Verificar uso de recursos:
   `docker stats diagnostico --no-stream`
5. Ver alterações no filesystem:
   `docker diff diagnostico`

Questões:

- No resultado do docker inspect, onde você encontraria o endereço IP interno do container?

O ip estará no campo (NetworkSettings.IPAddress)

- O que as letras A, C e D significam na saída do docker diff?

A - Arquivo adicionado

C - Arquivo modificado

D- Arquivo removido

- Qual comando você usaria para acompanhar os logs em tempo real enquanto acessa a aplicação pelo navegador?

docker logs -f diagnostico

- O docker stats mostrou o limite de memória do container. Como você definiria um limite de 128MB ao criar o container?

utilizando a flag -m 128m, desta forma limitando a memória do container

Referência de Comandos — Resumo

Tabela de referência rápida dos comandos utilizados nos exercícios:

Comando	Exemplo	Descrição
`docker run -d`	`docker run -d nginx`	Executa em background
`docker run -it`	`docker run -it ubuntu bash`	Terminal interativo
`docker run --name`	`docker run --name web nginx`	Define nome
`docker run -p`	`docker run -p 8080:80 nginx`	Mapeia porta host:container
`docker run -e`	`docker run -e SENHA=123 app`	Define variável de ambiente
`docker run --rm`	`docker run --rm ubuntu echo oi`	Remove ao finalizar
`docker ps`	`docker ps`	Lista containers ativos
`docker ps -a`	`docker ps -a`	Lista todos containers
`docker stop`	`docker stop meu-nginx`	Para container (graceful)
`docker kill`	`docker kill meu-nginx`	Para container (forçado)
`docker start`	`docker start meu-nginx`	Inicia container parado
`docker restart`	`docker restart meu-nginx`	Reinicia container
`docker pause`	`docker pause meu-nginx`	Pausa processos
`docker unpause`	`docker unpause meu-nginx`	Retoma processos
`docker rm`	`docker rm meu-nginx`	Remove container parado
`docker rm -f`	`docker rm -f meu-nginx`	Remove em execução
`docker container prune`	`docker container prune`	Remove todos parados
`docker logs`	`docker logs meu-nginx`	Exibe logs
`docker logs -f`	`docker logs -f meu-nginx`	Acompanha logs ao vivo
`docker exec`	`docker exec web ls /`	Executa comando no container
`docker exec -it`	`docker exec -it web bash`	Abre terminal no container
`docker inspect`	`docker inspect meu-nginx`	Detalhes completos (JSON)
`docker stats`	`docker stats --no-stream`	Uso de CPU/memória
`docker top`	`docker top meu-nginx`	Processos do container
`docker diff`	`docker diff meu-nginx`	Alterações no filesystem
`docker cp`	`docker cp web:/etc/nginx .`	Copia arquivos host↔container

Bons estudos!